ENGLISH-TO-FRENCH DEEP LEARNING TRANSLATION PROJECT

Project Report • NLP • Pretrained Transformer • Streamlit

Scope: English-to-French translation, original LSTM experiment, Hugging Face Marian model, related NLP concepts, evaluation, and deployment considerations. Image classification is excluded.

Executive Summary

This project explores an English-to-French translation application using deep learning and natural language processing (NLP). An initial sequence-to-sequence LSTM model was built and saved with its vectorizers. Because its generated translations were unreliable, the application's inference layer was changed to a pretrained model from the Hugging Face Hub: **Helsinki-NLP/opus-mt-en-fr**, based on the Marian encoder–decoder Transformer architecture. The Python function in **translator.py** loads the tokenizer and model, generates French output, and is called by the Streamlit interface in **app.py**.

The current translation path uses pretrained weights for inference. The information available does not establish that Marian has been fine-tuned on local **fra.txt**, that formal benchmark results have been recorded, or that Streamlit Community Cloud deployment is complete. Sentence embeddings, cosine similarity, BERT concepts, activation functions, and prompting are included as related learning/evaluation topics, not as features claimed to be implemented in the current translation path.

1. Problem Statement

Training a translation model from scratch requires aligned bilingual data, correct sequence alignment, sufficient training, and careful evaluation. The original LSTM produced semantically incorrect output for some test inputs. The practical change was to use a pretrained translation model and integrate it into the application, avoiding another full training run just to obtain a usable baseline.

2. Objectives

- Accept English text and display a French translation through a simple application interface.
- Explain the NLP pipeline: tokenization, model inference, and decoding.
- Understand the difference between a sequence-to-sequence LSTM and an encoder–decoder Transformer.
- Define an evaluation plan using test examples, reference-based metrics, latency, and model size.
- Document the requirements for local operation and possible cloud deployment.

3. Architecture and Workflow

Implemented flow: English text entered in Streamlit → **app.py** calls **translate_sentence()** in **translator.py** → Marian tokenizer converts text into model input tokens → pretrained Marian model generates French token IDs → tokenizer decodes those IDs into readable French → Streamlit displays the translation.

Component	Role	Status
Streamlit / app.py	User interface for entering text and viewing translation.	Local app reported working; cloud deployment not verified.
translator.py	Loads model/tokenizer and implements inference.	Updated to use MarianTokenizer and MarianMTModel.
Hugging Face Transformers	Python library to load and run supported pretrained models.	Used.
Helsinki-NLP/opus-mt-en-fr	Pretrained English-to-French Marian translation model.	Used for inference; fine-tuning not established.
Original LSTM files	Earlier encoder–decoder experiment and possible baseline.	Kept locally; current translator no longer uses them.

4. Model Approaches

4.1 Original LSTM encoder–decoder

The encoder LSTM read English token IDs and represented the sequence using hidden and cell states. The decoder used these states and preceding French tokens to predict subsequent French tokens. TextVectorization layers converted sentences into integer sequences. Training used shifted French sequences: decoder input contains the preceding tokens and the expected output contains the next tokens. Despite this structure, observed translations were poor, so the inference engine was changed.

4.2 Pretrained Marian Transformer

The current model is **Helsinki-NLP/opus-mt-en-fr**, a pretrained Marian encoder–decoder Transformer for English-to-French translation. It is loaded with **MarianTokenizer.from_pretrained()** and **MarianMTModel.from_pretrained()**. The model already contains learned parameters. The current workflow uses it for inference, not for training from scratch or fine-tuning.

4.3 Why use a pretrained model?

A pretrained translation model is a stronger starting point for general translation than an underperforming small model trained from scratch. It avoids training all parameters on a local dataset before the application can be used. It does not guarantee perfect translations; representative testing and evaluation are still necessary.

5. Important Keywords and Purpose

Keyword	Meaning / role
NLP	Natural Language Processing: methods for processing and generating human language.
Tokenization / decoding	Converts text to token IDs and generated IDs back into readable French.
LSTM	Recurrent neural network with hidden and cell states; used in the original model.
BERT	Transformer encoder for contextual text understanding; not the current translation model.
Transformer / Marian	Attention-based encoder–decoder architecture used by the current translation model.
Sentence embeddings	Dense sentence vectors that can support a separate semantic comparison experiment.
Cosine similarity	Compares vector directions; useful for embeddings, not a translation accuracy metric.
ReLU / Leaky ReLU	Nonlinear activation functions often used in hidden layers.
Sigmoid / tanh	Sigmoid maps a scalar to 0–1; tanh maps to –1–1 and is used in recurrent calculations.
Softmax	Converts logits into a probability distribution over output candidates or tokens.
Prompting	Instructions given to a generative LLM; not required for the current Marian translation API.
BLEU / chrF	Reference-based translation metrics; should be paired with human review.
Inference latency / model size	Measures response time and storage footprint; values must be measured, not guessed.

6. Evaluation Plan

The user reports that local translations are now mostly correct. No independently verified aggregate translation score, BLEU/chrF result, latency benchmark, or controlled LSTM-versus-Marian comparison has been supplied. Numerical results should therefore be measured rather than invented.

- **Test set:** prepare 15–20 varied English sentences, including questions, longer text, punctuation, and unfamiliar words; review meaning and grammar.
- **BLEU / chrF:** compare generated output with reference French translations; interpret scores alongside human review.
- **Latency:** measure per-sentence response time after separating first-load/model-download time.
- **Model size/resources:** record weight-file size and the machine's RAM/CPU/GPU context.
- **Fair comparison:** if comparing LSTM and Marian, use the same held-out examples and evaluation method.

7. Streamlit Deployment

A deployment repository should include **app.py**, **translator.py**, **requirements.txt**, and **README.md**. Dependencies need compatible versions of Streamlit, PyTorch, Transformers 4.x, SentencePiece, and Sacremoses. The cloud instance must be able to download the model on first startup and fit it in available memory. Deployment should only be described as complete after the public app URL has been tested.

8. Limitations and Future Work

- Translations can still be inaccurate or awkward; no model guarantees perfect output.
- Fine-tuning Marian on fra.txt is not part of the workflow described here.
- A controlled benchmark is needed before claiming that one model is more accurate or faster.
- Sentence embeddings and cosine similarity can be added as a separate NLP feature, but are not part of the current translation path unless implemented.
- Future work: evaluation, robust error handling, verified cloud deployment, and optional domain-specific fine-tuning.

9. Conclusion

The project demonstrates English-to-French translation using a pretrained Marian Transformer integrated with Python and Streamlit. The original LSTM experiment provides learning context and a potential baseline, while Marian currently performs inference. This report distinguishes working functionality from proposed evaluation and future work.

References: https://huggingface.co/Helsinki-NLP/opus-mt-en-fr • https://huggingface.co/docs/transformers/ • https://docs.streamlit.io/deploy/streamlit-community-cloud